Real-world — a support-reply router — Solution

Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `runnable_routing_practice.py`: a real support desk doesn't answer every ticket the same way — a billing question, a bug report, and a random hello each need a different reply. This exercise builds exactly that: one reply chain per category, a `RunnableBranch` that routes each ticket to the right one, and a `RunnableLambda` that tags the result. **If not:** Project 6's ticket-triage pipeline (and Doc11's Supervisor) would be the first time you met routing with `RunnableBranch`, in a much bigger file.

Basic Version

Approach 1 — three reply chains and a branch

```
# runnable_routing_practice.py
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnableBranch, RunnableLambda
from langchain_openai import ChatOpenAI

BILLING = "You are a billing agent. Be brief; mention the refund policy."
TECHNICAL = "You are a tech-support agent. Ask for steps to reproduce."
GENERAL = "You are a general support agent. Be friendly and brief."


def reply_chain(model, system_text):
    prompt = ChatPromptTemplate.from_messages(
        [("system", system_text), ("human", "{text}")]
    )
    return prompt | model | StrOutputParser()


def main():
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0, timeout=30)
    router = RunnableBranch(
        (lambda t: t["category"] == "billing", reply_chain(model, BILLING)),
        (lambda t: t["category"] == "technical", reply_chain(model, TECHNICAL)),
        reply_chain(model, GENERAL),          # default — no condition
    )
```

```
    tag = RunnableLambda(lambda text: f"{text.strip()} (auto-routed)")
    pipeline = router | tag

    tickets = [
        {"category": "billing", "text": "I was charged twice."},
        {"category": "technical", "text": "The app crashes on login."},
        {"category": "hello", "text": "Just saying hi!"},
    ]
    for ticket in tickets:
        print(ticket["category"], "->", pipeline.invoke(ticket))


if __name__ == "__main__":
    main()
```

The default (the last argument with no condition) is what makes the unknown `"hello"` ticket work instead of raising — never leave it out.

Intermediate Version

Approach 2 — named builders + an offline routing test

Each piece gets its own small function, so the router is reusable and you can prove the routing is correct **without a model or a key** — a fake model that echoes the system line it received tells you which branch ran.

```
# runnable_routing_practice.py
from langchain_core.messages import AIMessage
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnableBranch, RunnableLambda

BILLING = "You are a billing agent. Be brief; mention the refund policy."
TECHNICAL = "You are a tech-support agent. Ask for steps to reproduce."
GENERAL = "You are a general support agent. Be friendly and brief."


def build_reply_chain(model, system_text):
    prompt = ChatPromptTemplate.from_messages(
        [("system", system_text), ("human", "{text}")]
    )
    return prompt | model | StrOutputParser()


def build_router(model):
```

```
    billing = build_reply_chain(model, BILLING)
    technical = build_reply_chain(model, TECHNICAL)
    general = build_reply_chain(model, GENERAL)
    return RunnableBranch(
        (lambda t: t["category"] == "billing", billing),
        (lambda t: t["category"] == "technical", technical),
        general,
    )


def tag_reply(text):
    return f"{text.strip()} (auto-routed)"


def build_pipeline(model):
    return build_router(model) | RunnableLambda(tag_reply)
```

The real `main()` uses a real model:

```
def main():
    from langchain_openai import ChatOpenAI
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0, timeout=30)
    pipeline = build_pipeline(model)
    for category in ["billing", "technical", "hello"]:
        reply = pipeline.invoke({"category": category, "text": "help me"})
        print(f"{category} -> {reply}")
```

The tests prove routing with a fake model (no key, no cost):

```
def fake_echo_model():
    # a 'model' that returns the system line it was given, so the reply
    # tells us which branch ran
    def pick(prompt_value):
        system_line = prompt_value.to_messages()[0].content
        return AIMessage(content=system_line)
    return RunnableLambda(pick)


def test_each_category_routes_right():
    pipeline = build_pipeline(fake_echo_model())
    billing = pipeline.invoke({"category": "billing", "text": "x"})
    technical = pipeline.invoke({"category": "technical", "text": "x"})
    other = pipeline.invoke({"category": "sales", "text": "x"})
    result = "OK"
    if "billing agent" not in billing:
        result = "WRONG"
```

```
    if "tech-support agent" not in technical:
        result = "WRONG"
    if "general support agent" not in other:
        result = "WRONG"
    print(f"case 1 (each category -> right branch) -> {result}")


def test_unknown_uses_default_and_tags():
    pipeline = build_pipeline(fake_echo_model())
    out = pipeline.invoke({"category": "anything-else", "text": "x"})
    result = "OK"
    if "general support agent" not in out:
        result = "WRONG"
    if "(auto-routed)" not in out:
        result = "WRONG"
    print(f"case 2 (unknown -> default + tagged)   -> {result}")


if __name__ == "__main__":
    test_each_category_routes_right()
    test_unknown_uses_default_and_tags()
```

Expected output (the tests, no key needed):

```
case 1 (each category -> right branch) -> OK
case 2 (unknown -> default + tagged)   -> OK
```

Difference between Basic and Intermediate: Basic wires the router inline and runs it. Intermediate splits it into `build_reply_chain` / `build_router` / `build_pipeline`, so the routing can be tested offline with a fake model — exactly how Project 6 proves its triage routes correctly before spending a token.

What you proved: the three chain shapes together — sequential (`|`), your own function (`RunnableLambda`), and conditional (`RunnableBranch`) — in one real pipeline, with the routing checked for free.